Procesamiento funcional de datos para un sistema de gestión de transporte urbano

Actividad evaluativa EA2: Creación de repositorio

Eduardo Stivens Cano Peña

Melanny Natalia Murillo Muñoz

Institución Universitaria Digital de Antioquia

Programación Orientada a Objetos 2

Docente: Leonardo Montes Gutierrez

31-10-2026

Introducción

La Alcaldía de TecnoValle necesita modernizar la forma en que analiza los millones de registros que genera su sistema de transporte cada día. La plataforma actual, escrita con un estilo imperativo, es lenta, comparte estructuras mutables entre procesos y produce conteos erróneos cuando varias tareas escriben a la vez. Por eso el proyecto TecnoMovil Data plantea migrar el módulo de agregación diaria hacia un enfoque funcional.

En este trabajo se desarrolló, en Java sin frameworks externos, un módulo que resuelve seis consultas sobre los eventos de entrada y salida de los usuarios: afluencia por estación, horas pico, rutas más utilizadas, patrones de viaje, tiempo promedio entre estaciones y detección de rutas sobrecargadas. Para lograrlo se emplearon expresiones lambda, funciones puras, colecciones inmutables y pipelines de Streams.

El documento presenta primero los objetivos, luego el código fuente con la explicación de las decisiones de diseño, después la evidencia de ejecución y, por último, las conclusiones y los enlaces a los repositorios del grupo.

Objetivos

Objetivo general

Aplicar conceptos de programación funcional en Java para procesar de manera declarativa un gran volumen de registros de movilidad urbana simulados.

Objetivos específicos

- Implementar las seis operaciones solicitadas mediante Streams y lambdas, sin modificar las colecciones originales.
- Diseñar funciones puras que dependan solo de sus parámetros y que no produzcan efectos secundarios.
- Comprobar que el procesamiento puede ejecutarse en paralelo con parallelStream() y entrega el mismo resultado que la versión secuencial.
- Alojar la solución en la rama principal de un repositorio de GitHub y documentar la evidencia de ejecución.

Desarrollo

Estructura del proyecto

La solución se compone de tres clases, todas en el paquete por defecto: RegistroTransito, que modela un evento; ProcesadorMovilidad, que reúne las seis operaciones de análisis; y Main, que simula los datos y muestra los resultados. No se utilizó ninguna librería externa; solo clases de java.time, java.util y java.util.stream.

Código fuente

A continuación, se incluye el código completo de cada archivo.

Tabla 1

Clase RegistroTransito

```
import java.time.LocalDateTime;

// Se utiliza 'record' para garantizar la inmutabilidad absoluta de cada evento del sistema.
public record RegistroTransito(
    String idUsuario,
    String ruta,
    String estacion,
    String accion,
    LocalDateTime timestamp
) {}
```

Tabla 2

Clase ProcesadorMovilidad

```
import java.time.Duration;
import java.util.Comparator;
import java.util.List;
import java.util.Map;
import java.util.stream.Collectors;
import java.util.stream.IntStream;

public class ProcesadorMovilidad {

    // a) Afluencia por estación: solo cuenta los eventos de "entrada".
    public Map<String, Long> calcularAfluencia(List<RegistroTransito> registros) {
```

```
    return registros.stream()
            .filter(r -> r.accion().equalsIgnoreCase("entrada"))
            .collect(Collectors.groupingBy(
                    RegistroTransito::estacion,
                    Collectors.counting()));
}

// a) Variante paralela: mismo resultado, procesada con parallelStream().
public Map<String, Long> calcularAfluenciaParalela(List<RegistroTransito> registros) {
    return registros.parallelStream()
            .filter(r -> r.accion().equalsIgnoreCase("entrada"))
            .collect(Collectors.groupingBy(
                    RegistroTransito::estacion,
                    Collectors.counting()));
}

// b) Horas pico: registros agrupados por hora y ordenados de mayor a menor flujo.
public List<Map.Entry<Integer, Long>> identificarHorasPico(List<RegistroTransito> registros) {
    return registros.stream()
            .collect(Collectors.groupingBy(
                    r -> r.timestamp().getHour(),
                    Collectors.counting()))
            .entrySet().stream()
            .sorted(Map.Entry.<Integer, Long>comparingByValue().reversed()
                    .thenComparing(Map.Entry.comparingByKey()))
            .toList();
}

// c) Rutas más utilizadas, de mayor a menor volumen.
public List<Map.Entry<String, Long>> rutasMasUtilizadas(List<RegistroTransito> registros) {
    return registros.stream()
            .collect(Collectors.groupingBy(
                    RegistroTransito::ruta,
                    Collectors.counting()))
            .entrySet().stream()
            .sorted(Map.Entry.<String, Long>comparingByValue().reversed()
                    .thenComparing(Map.Entry.comparingByKey()))
            .toList();
}

// d) Patrones de viaje: estaciones por usuario en orden cronológico.
public Map<String, List<String>> obtenerPatronesDeViaje(List<RegistroTransito> registros) {
    return registros.stream()
            .sorted(Comparator.comparing(RegistroTransito::timestamp))
            .collect(Collectors.groupingBy(
                    RegistroTransito::idUsuario,
                    Collectors.mapping(
                            RegistroTransito::estacion,
```

```
                        Collectors.toUnmodifiableList())));
    }

    // e) Tiempo promedio (minutos) entre registros consecutivos de un mismo usuario.
    public double tiempoPromedioEntreEstaciones(List<RegistroTransito> registros) {
        Map<String, List<RegistroTransito>> viajesPorUsuario = registros.stream()
                .sorted(Comparator.comparing(RegistroTransito::timestamp))
                .collect(Collectors.groupingBy(RegistroTransito::idUsuario));

        return viajesPorUsuario.values().stream()
                .flatMap(viajes -> IntStream.range(0, viajes.size() - 1)
                        .mapToObj(i -> Duration.between(
                                viajes.get(i).timestamp(),
                                viajes.get(i + 1).timestamp()).toMinutes()))
                .mapToLong(Long::longValue)
                .average()
                .orElse(0.0);
    }

    // f) Rutas críticas: las que alcanzan o superan el umbral recibido por parámetro.
    public List<String> detectarSobrecarga(List<RegistroTransito> registros, long umbralCritico) {
        return registros.stream()
                .collect(Collectors.groupingBy(RegistroTransito::ruta, Collectors.counting()))
                .entrySet().stream()
                .filter(entry -> entry.getValue() >= umbralCritico)
                .map(Map.Entry::getKey)
                .sorted()
                .toList();
    }
}
```

Tabla 3

Clase Main

```
import java.time.LocalDateTime;
import java.util.List;
import java.util.stream.IntStream;

public class Main {

    // Genera registros simulados de forma determinista (sin estado compartido).
    private static List<RegistroTransito> generarLote(int cantidad) {
        List<String> rutas = List.of("Ruta_A", "Ruta_B", "Ruta_C", "Ruta_D");
        List<String> estaciones = List.of("Estacion_Norte", "Estacion_Centro",
                "Estacion_Sur", "Estacion_Occidente", "Estacion_Oriente");
        LocalDateTime inicio = LocalDateTime.of(2026, 10, 31, 5, 0);

        return IntStream.range(0, cantidad)
```

```
            .mapToObj(i -> new RegistroTransito(
                    "U" + (i % 5000),
                    rutas.get((i * 7) % rutas.size()),
                    estaciones.get((i * 3) % estaciones.size()),
                    i % 2 == 0 ? "entrada" : "salida",
                    inicio.plusSeconds((long) i * 3)))
            .toList();
    }
    public static void main(String[] args) {
        // Lote pequeño e inmutable (List.of no permite modificaciones)
        List<RegistroTransito> loteDatos = List.of(
            new RegistroTransito("U001", "Ruta_A", "Estacion_Norte", "entrada",
                LocalDateTime.of(2026, 10, 31, 7, 15)),
            new RegistroTransito("U001", "Ruta_A", "Estacion_Centro", "salida",
                LocalDateTime.of(2026, 10, 31, 7, 45)),
            new RegistroTransito("U002", "Ruta_B", "Estacion_Sur", "entrada",
                LocalDateTime.of(2026, 10, 31, 8, 0)),
            new RegistroTransito("U003", "Ruta_A", "Estacion_Norte", "entrada",
                LocalDateTime.of(2026, 10, 31, 7, 20)),
            new RegistroTransito("U002", "Ruta_B", "Estacion_Occidente", "salida",
                LocalDateTime.of(2026, 10, 31, 8, 30))
        );

        ProcesadorMovilidad procesador = new ProcesadorMovilidad();

        System.out.println("=== LOTE PEQUEÑO (5 registros) ===");

        System.out.println("\n--- a) Afluencia por Estación ---");
        procesador.calcularAfluencia(loteDatos).forEach((est, n) -> System.out.println(est + ": " + n));

        System.out.println("\n--- b) Horas Pico ---");
        procesador.identificarHorasPico(loteDatos)
                .forEach(e -> System.out.println(e.getKey() + "h: " + e.getValue()));

        System.out.println("\n--- c) Rutas Más Utilizadas ---");
        procesador.rutasMasUtilizadas(loteDatos)
                .forEach(e -> System.out.println(e.getKey() + ": " + e.getValue()));

        System.out.println("\n--- d) Patrones de Viaje del Usuario U001 ---");
        System.out.println(procesador.obtenerPatronesDeViaje(loteDatos).get("U001"));

        System.out.println("\n--- e) Tiempo Promedio entre Estaciones (Minutos) ---");
        System.out.println(procesador.tiempoPromedioEntreEstaciones(loteDatos));

        System.out.println("\n--- f) Rutas en Sobrecarga (Umbral >= 2) ---");
        System.out.println(procesador.detectarSobrecarga(loteDatos, 2));

        // Lote grande: demuestra escalabilidad y paralelización
```

```
        List<RegistroTransito> loteGrande = generarLote(1_000_000);
        System.out.println("\n=== LOTE GRANDE (" + loteGrande.size() + " registros) ===");

        long t1 = System.nanoTime();
        var secuencial = procesador.calcularAfluencia(loteGrande);
        long t2 = System.nanoTime();
        var paralelo = procesador.calcularAfluenciaParalela(loteGrande);
        long t3 = System.nanoTime();

        System.out.println("Afluencia (secuencial): " + secuencial);
        System.out.println("Tiempo secuencial: " + (t2 - t1) / 1_000_000 + " ms");
        System.out.println("Tiempo paralelo:   " + (t3 - t2) / 1_000_000 + " ms");
        System.out.println("¿Resultados idénticos? " + secuencial.equals(paralelo));
        System.out.println("Rutas críticas (umbral 250000): "
                + procesador.detectarSobrecarga(loteGrande, 250_000));
        System.out.println("Hora pico principal: "
                + procesador.identificarHorasPico(loteGrande).get(0));
    }
}
```

Explicación de los conceptos funcionales aplicados

Funciones puras

Cada método de ProcesadorMovilidad es una función pura: su resultado depende únicamente de la lista y del umbral que recibe, no lee ni escribe variables de la clase y no imprime ni altera nada fuera de sí mismo. Con la misma entrada siempre devuelve la misma salida. Por ejemplo, detectarSobrecarga recibe el umbral como parámetro en lugar de guardarlo en un atributo, lo que evita depender de un estado externo. Incluso el generador de datos de Main calcula cada registro a partir de su índice, sin usar un generador aleatorio compartido.

Inmutabilidad

RegistroTransito está declarado como record, por lo que sus campos son finales y no existen métodos de modificación. Los datos de prueba se crean con List.of, que produce una lista no modificable, y las consultas devuelven nuevas colecciones mediante toList() y Collectors.toUnmodifiableList(). Ningún método altera la lista original: al ordenar por fecha, sorted trabaja sobre el flujo y no sobre la colección de origen.

Funciones de orden superior

Varias operaciones reciben otras funciones como argumento. filter recibe un predicado, map y mapToObj reciben funciones de transformación, sorted recibe un comparador y groupingBy recibe la función que define la clave de agrupación. Los comparadores se construyen componiendo funciones con comparingByValue().reversed().thenComparing(...), lo que además hace que los empates se resuelvan siempre igual.

Expresiones lambda y referencias a métodos

Las lambdas expresan en una línea lo que antes requeriría un ciclo con condiciones, por ejemplo r -> r.accion().equalsIgnoreCase("entrada") o r -> r.timestamp().getHour(). Cuando la lambda solo invoca un método existente se usó la referencia equivalente, como RegistroTransito::estacion o Map.Entry::getKey, que resulta más legible.

Composición de funciones

La composición de funciones aparece en dos niveles. Por un lado, cada consulta es una cadena de etapas pequeñas en la que la salida de una alimenta a la siguiente; en detectarSobrecarga, por ejemplo, se encadenan groupingBy, filter, map y sorted sin variables intermedias que se modifiquen. Por otro lado, las propias funciones se combinan entre sí: comparingByValue().reversed().thenComparing(comparingByKey()) construye un comparador nuevo a partir de otros dos, y Comparator.comparing(RegistroTransito::timestamp) convierte una función de extracción en un criterio de orden.

Streams

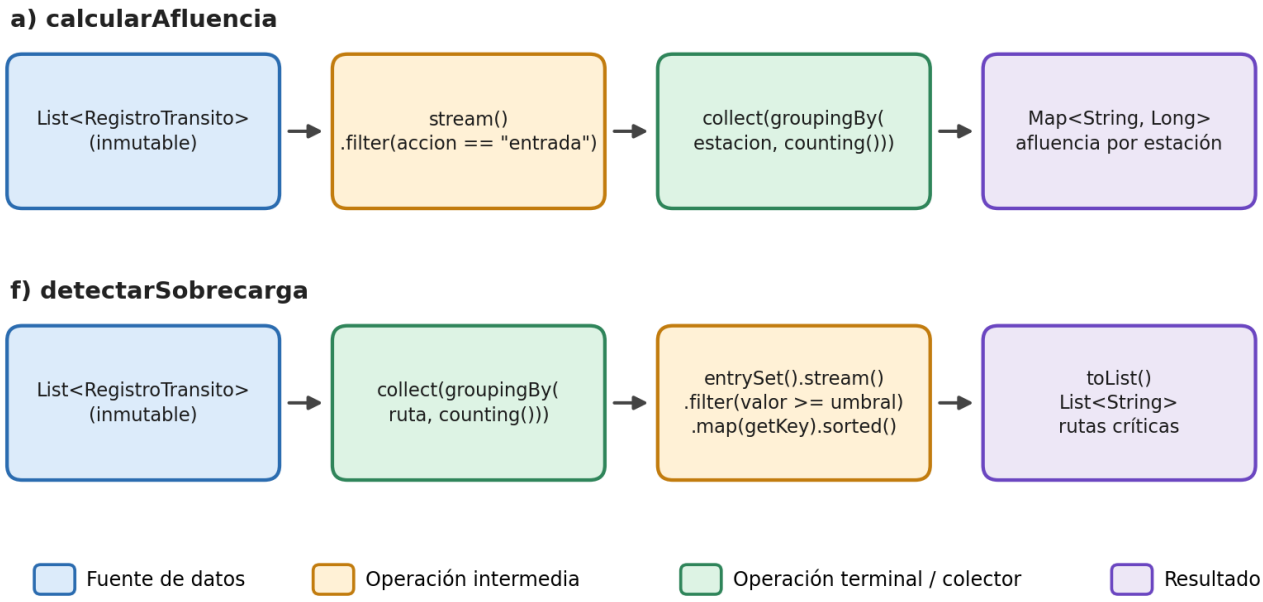
Todas las consultas son pipelines de tres etapas: una fuente (stream() o IntStream.range), operaciones intermedias (filter, sorted, map, flatMap) y una operación terminal (collect, toList, average). El colector groupingBy con counting reemplaza los contadores manuales y los mapas mutables del sistema anterior, que eran la causa de las condiciones de carrera descritas en el caso. Para el tiempo promedio se usó IntStream.range

sobre cada lista de viajes con el fin de comparar cada registro con el siguiente, y average() devuelve un OptionalDouble que se resuelve con orElse(0.0) para el caso sin datos.

La Figura 1 muestra cómo se componen estas etapas en dos de las consultas: la fuente es siempre una lista inmutable, las operaciones intermedias transforman el flujo sin modificarla y la operación terminal produce un resultado nuevo.

Figura 1

Pipelines funcionales de calcularAfluencia y detectarSobrecarga



Paralelización

Como los métodos no comparten estado, es posible cambiar stream() por parallelStream() sin riesgo de conteos incorrectos. Se añadió calcularAfluenciaParalela para demostrarlo: en Main se ejecutan ambas versiones sobre un millón de registros simulados y se verifica que los resultados sean idénticos. Las operaciones que dependen del orden de los eventos (patrones de viaje y tiempo promedio) ordenan primero por timestamp, de modo que el resultado no depende de cómo se reparta el trabajo.

Resumen de las operaciones

Tabla 4

Operaciones implementadas y técnicas empleadas

Operación	Método	Técnica principal
a) Afluencia por estación	calcularAfluencia	filter + groupingBy + counting
b) Horas pico	identificarHorasPico	groupingBy por getHour + sorted descendente
c) Rutas más utilizadas	rutasMasUtilizadas	groupingBy por ruta + sorted descendente
d) Patrones de viaje	obtenerPatronesDeViaje	sorted por fecha + groupingBy + mapping
e) Tiempo promedio	tiempoPromedioEntreEstaciones	IntStream.range + flatMap + average
f) Sobrecarga	detectarSobrecarga	groupingBy + filter con umbral parametrizado

Evidencia de ejecución

Con el lote de cinco registros, los resultados coinciden con el cálculo manual. Entran dos usuarios por Estacion_Norte y uno por Estacion_Sur; el usuario U001 recorre Estacion_Norte y luego Estacion_Centro; el tiempo promedio entre registros es de 30 minutos; y con un umbral de 2 las rutas Ruta_A (3 registros) y Ruta_B (2 registros) quedan marcadas como críticas.

La Figura 2 también incluye la ejecución con el lote grande de un millón de registros. Cada una de las cinco estaciones registró 100.000 entradas y las cuatro rutas alcanzaron 250.000 registros, por lo que todas superan el umbral crítico definido. En las horas pico hay varias horas empatadas con 42.000 registros y el criterio de desempate elige la menor, es decir, las 5. La versión secuencial tardó 82 ms y la paralela 75 ms: con este volumen la diferencia es pequeña, pero ambas entregaron resultados idénticos, lo que confirma que el código es seguro para paralelizar.

Figura 2

Ejecución del programa en Visual Studio Code

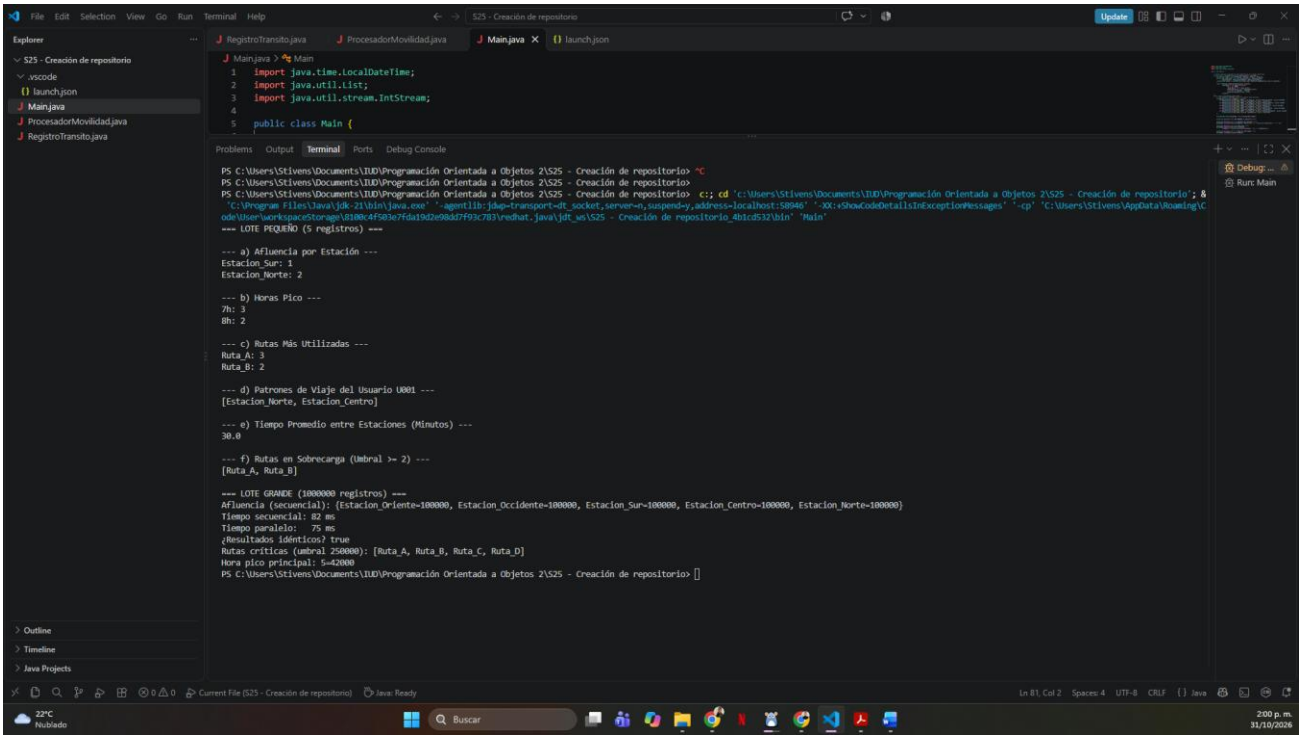
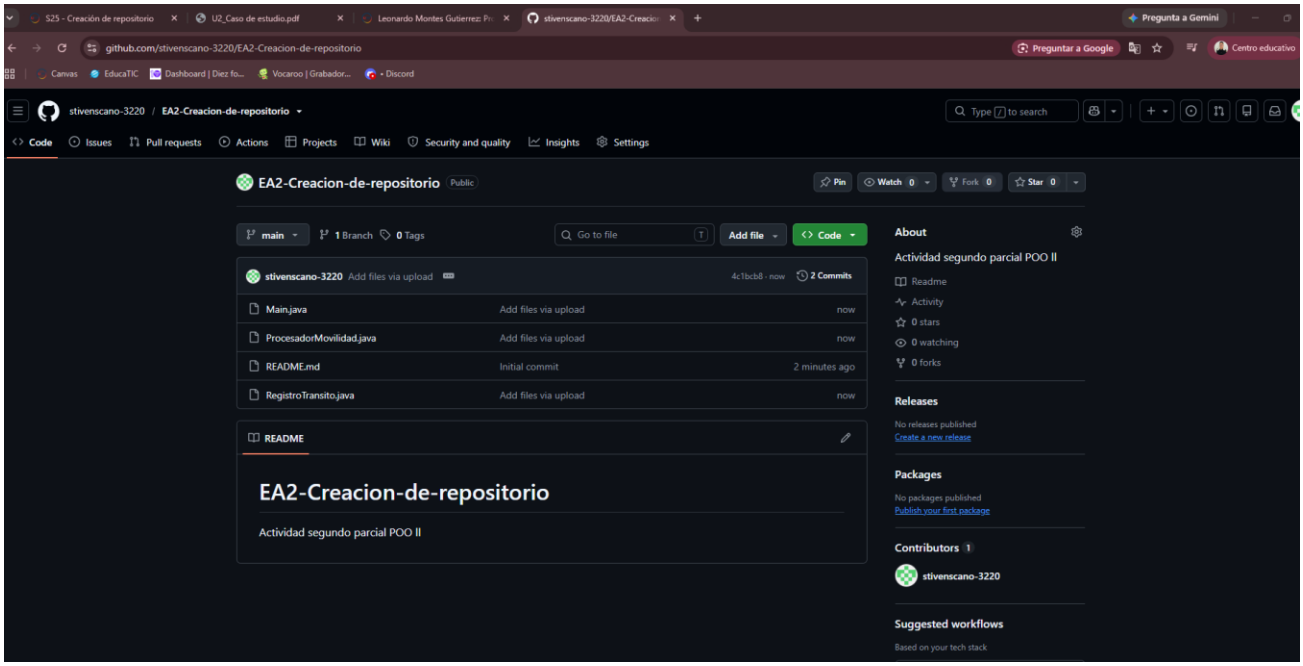
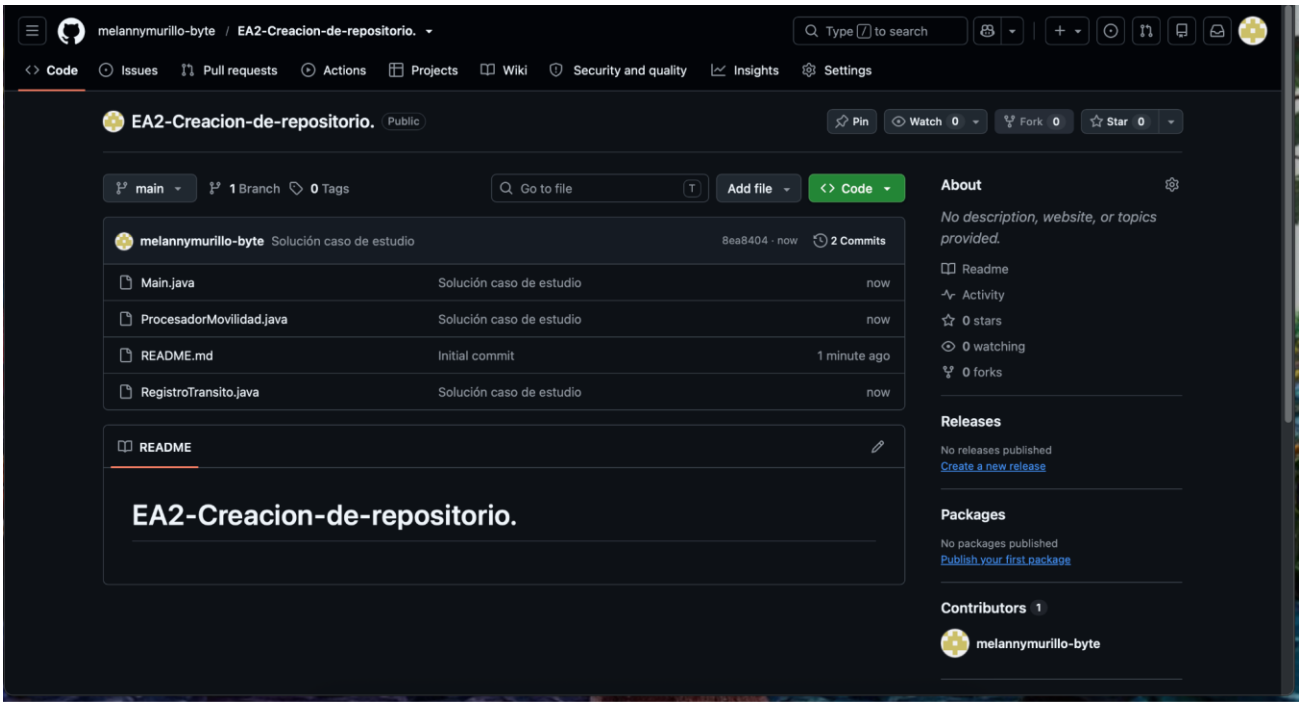




Figura 3

Estructura del repositorio en GitHub, rama principal




melannymurillo-byte / EA2-Creacion-de-repositorio.
Type / to search
Code
Issues
Pull requests
Actions
Projects
Wiki
Security and quality
Insights
Settings
EA2-Creacion-de-repositorio.
Public
Pin
Watch 0
Fork 0
Star 0
main
1 Branch
0 Tags
Go to file
T
Add file
Code
melannymurillo-byte Solución caso de estudio
8ea8404 · now
2 Commits
Main.java
Solución caso de estudio
now
ProcesadorMovilidad.java
Solución caso de estudio
now
README.md
Initial commit
1 minute ago
RegistroTransito.java
Solución caso de estudio
now
README
EA2-Creacion-de-repositorio.
About
No description, website, or topics provided.
Readme
Activity
0 stars
0 watching
0 forks
Releases
No releases published
Create a new release
Packages
No packages published
Publish your first package
Contributors 1
melannymurillo-byte

Conclusiones

Las seis consultas del caso se resolvieron con pipelines declarativos de pocas líneas, sin ciclos ni contadores manuales. Esto reduce el código que hay que mantener y deja a la vista qué se calcula en lugar de cómo se recorre la colección, que era justamente uno de los problemas de la plataforma actual.

El uso de un record, de listas no modificables y de métodos sin estado eliminó la causa de los conteos erróneos: al no existir estructuras compartidas que se modifiquen, tampoco hay condiciones de carrera. Esa misma propiedad permite paralelizar el análisis con parallelStream() y obtener el mismo resultado que en modo secuencial, lo cual es relevante para volúmenes de entre 6 y 12 millones de registros diarios.

Durante el desarrollo se corrigieron errores de tipos genéricos en los métodos que devuelven listas de entradas de mapa y se agregó un criterio de desempate en los ordenamientos para que los resultados sean reproducibles. Como mejora futura, el campo accion podría modelarse como un enum en lugar de una cadena para evitar errores de escritura, y el cálculo del tiempo promedio podría distinguir los trayectos entre estaciones de los intervalos entre viajes distintos del mismo usuario.

Enlaces de los repositorios

Eduardo Stivens Cano Peña:

https://github.com/stivenscano-3220/EA2-Creacion-de-repositorio

Melanny Natalia Murillo:

https://github.com/melannymurillo-byte/EA2-Creacion-de-repositorio..git

Referencias

Oracle. (s. f.). Package java.util.stream. Java Platform, Standard Edition 21 API Specification. https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/stream/package-summary.html

Urma, R.-G., Fusco, M., y Mycroft, A. (2019). Modern Java in action: Lambdas, streams, functional and reactive programming (2.ª ed.). Manning Publications.

Institución Universitaria Digital de Antioquia. (s. f.). Unidad 2: Caso de estudio [Material de curso]. Tecnología en Desarrollo de Software.